MALICK BUSS SÉNÉGAL

Marketplace E-Commerce Multi-Boutiques

Dossier d'Architecture Technique, Spécifications Opérationnelles & Guide de Déploiement Hybride

---

Auteurs :
Équipe Ingénierie & DevOps
Malick Buss Tech Core

Statut : Production Ready
Version : 1.0.0
Date : 2 novembre 2026

Plateforme Mobile-First • Transactions FCFA (Wave / Orange Money) • CI/CD & SonarQube

Dakar, République du Sénégal

Table des matières

1 Contexte Stratégique & Vision Produit **2**
1.1 Vision Globale & Piliers Fondamentaux 2
1.2 Acteurs & Modèle des Rôles (RBAC) 2

2 Architecture Logicielle & Modèle de Données **3**
2.1 Organisation du Monorepo 3
2.2 Modèle de Données Relationnel (PostgreSQL & Prisma) 3

3 Cartographie des Use Cases du Super Admin **4**
3.1 Synthèse des 8 Domaines Opérationnels 4

4 Sécurité, Qualité de Code & SonarQube **5**
4.1 Mesures de Sécurité Avancées 5
4.2 Contrôle Qualité & SonarQube Quality Gate 5

5 DevOps, Déploiement Hybride & CI/CD **7**
5.1 Stratégie Hybride : Render & Vercel 7
5.2 Sondes de Santé HTTP 7

6 Conclusion & Manuel d'Exploitation **9**
6.1 Commandes Usuelles pour l'Équipe 9

Chapitre 1

Contexte Stratégique & Vision Produit

1.1 Vision Globale & Piliers Fondamentaux

Malick Buss est une marketplace multi-boutiques mobile-first conçue spécifiquement pour répondre aux exigences structurelles du commerce digital au Sénégal (Dakar et régions de l'intérieur) et dans l'UEMOA.

La plateforme articule sa proposition de valeur autour de 4 piliers fondamentaux :

1. **Garantie de Confiance & Authenticité Produits :** Éradication de la contrefaçon et des photos trompeuses grâce à l'obligation de capture vidéo et photo en direct (*Direct Camera Live Capture*).
2. **Étanchéité Financière & Traçabilité :** Calcul automatique et transparent d'une commission marketplace de 10% par vente validée, avec isolation stricte des sous-commandes et journalisation comptable inviolable (*Commission Ledger*).
3. **Intégrations Locales en FCFA :** Support natif des solutions de paiement plébiscitées (Wave, Orange Money) et encadrement strict du Paiement à la Livraison (*Cash On Delivery*).
4. **Synergie WhatsApp Sécurisée :** Exploitation de WhatsApp pour la réassurance et l'assistance client, sans compromission des flux financiers de la marketplace.

1.2 Acteurs & Modèle des Rôles (RBAC)

— **Acheteur (CLIENT) :** Recherche géolocalisée, constitution d'un panier multi-boutiques unique, suivi d'expédition temps réel et réclamations sous 7 jours.

— **Vendeur / Marchand (VENDOR) :** Gestion autonome de la boutique, publication d'articles certifiés, traitement d'états des commandes et demande de retrait de solde vers Wave/Orange Money.

— **Super Administrateur (SUPER_ADMIN) :** Pouvoirs de gouvernance régalienne, modération de catalogue, vérification KYC des marchands, arbitrage des litiges et exécution des décaissements financiers.

Chapitre 2

Architecture Logicielle & Modèle de Données

2.1 Organisation du Monorepo

Le projet est architecturé sous la forme d'un monorepo modulaire garantissant la réutilisabilité du code et le typage strict de bout en bout :

- **`shared/`** : Bibliothèque partagée de types TypeScript, DTOs communs, utilitaires monétaires (`formatFCFA`) et formatage téléphonique international (`formatPhoneForWhatsApp`).
- **`backend/`** : API REST Core développée avec NestJS, validation DTO, authentification JWT, ORM Prisma et persistance PostgreSQL 16.
- **`frontend/`** : Application Web client et portail marchand développée en Next.js 14 App Router, optimisée pour les connexions mobiles 3G/4G.
- **`back-office/`** : Console de supervision et d'administration Super Admin (Next.js 14, Tailwind CSS, tableaux de bord de modération et gouvernance).

2.2 Modèle de Données Relationnel (PostgreSQL & Prisma)

La base de données relationnelle est orchestrée autour d'un schéma robuste :

- **Utilisateurs & Sécurité :** `users`, `roles`, `refresh_tokens`, `audit_logs`.
- **Boutiques & KYC :** `shops` avec statuts (`PENDING_VERIFICATION`, `ACTIVE`, `SUSPENDED`, `REJECTED`), coordonnées certifiées et toggle `requireLiveCapture`.
- **Catalogue Produits :** `products` avec gestion de stock, prix de base, prix remisé, conditions (`NEUF`, `TRES_BON_ETAT`) et URLs Cloudinary.
- **Commandes & Sous-Commandes :** `orders` parentes éclatées en `sub_orders` par marchand, permettant des cycles de vie indépendants (`PENDING_PAYMENT`, `TO_ACCEPT`, `ACCEPTED`, `PREPARING`, `SHIPPED`, `DELIVERED`, `COMPLETED`).
- **Grand Livre Financier :** `commission_ledgers` et `withdrawal_requests` pour la comptabilisation irréversible des gains et décaissements.

Chapitre 3

Cartographie des Use Cases du Super Admin

3.1 Synthèse des 8 Domaines Opérationnels

Domaine	Désignation	Responsabilités & Actions Clés
Domaine 1	Pilotage & KPI	Chiffre d'affaires global, volume des transactions, commissions perçues.
Domaine 2	Onboarding & KYC	Validation des dossiers marchands, activation et politique caméra directe.
Domaine 3	Relations Vendeurs	Annuaire unifié, contact direct WhatsApp wa.me/221..., suspension/réactivation.
Domaine 4	Modération Catalogue	Dépublication des articles non conformes, vérification anti-contrefaçon.
Domaine 5	Supervision Commandes	Suivi des sous-commandes, arbitrage des réclamations clients, remboursements.
Domaine 6	Gestion Financière	Validation des retraits marchands (seuil 5 000 FCFA), audit des virements Wave/OM.
Domaine 7	Sécurité & RBAC	Gestion des privilèges, désactivation de comptes suspects, journal d'audit.
Domaine 8	Gouvernance	Paramétrage du taux de commission (10%), frais de livraison (3 000 FCFA).

Table 3.1 – Matrice des responsabilités opérationnelles du Super Administrateur

Chapitre 4

Sécurité, Qualité de Code & SonarQube

4.1 Mesures de Sécurité Avancées

— **Chiffrement des Mots de Passe :** Dérivation de clé cryptographique via l'algorithme `scrypt` avec sel cryptographique de 16 octets et comparaison en temps constant (`timingSafeEqual`) pour prévenir les attaques temporelles.

— **En-têtes HTTP de Sécurité :** Injection systématique des en-têtes `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, `Referrer-Policy: strict-origin-when-cross-origin` et suppression de l'en-tête discriminant `X-Powered-By`.

— **Protection CORS Dynamique :** Filtrage strict des requêtes cross-origin avec support automatique des domaines de preview et de production Vercel (`*.vercel.app`) et des domaines officiels Malick Buss.

4.2 Contrôle Qualité & SonarQube Quality Gate

Le fichier de configuration `sonar-project.properties` assure une couverture complète :

```
sonar.projectKey=malick-buss
sonar.projectName=Malick Buss Marketplace
sonar.sources=backend/src,frontend/src,back-office/src,shared/src
sonar.tests=backend/test
sonar.javascript.lcov.reportPaths=backend/coverage/lcov.info
```

Listing 4.1 – Configuration SonarQube de l'entreprise

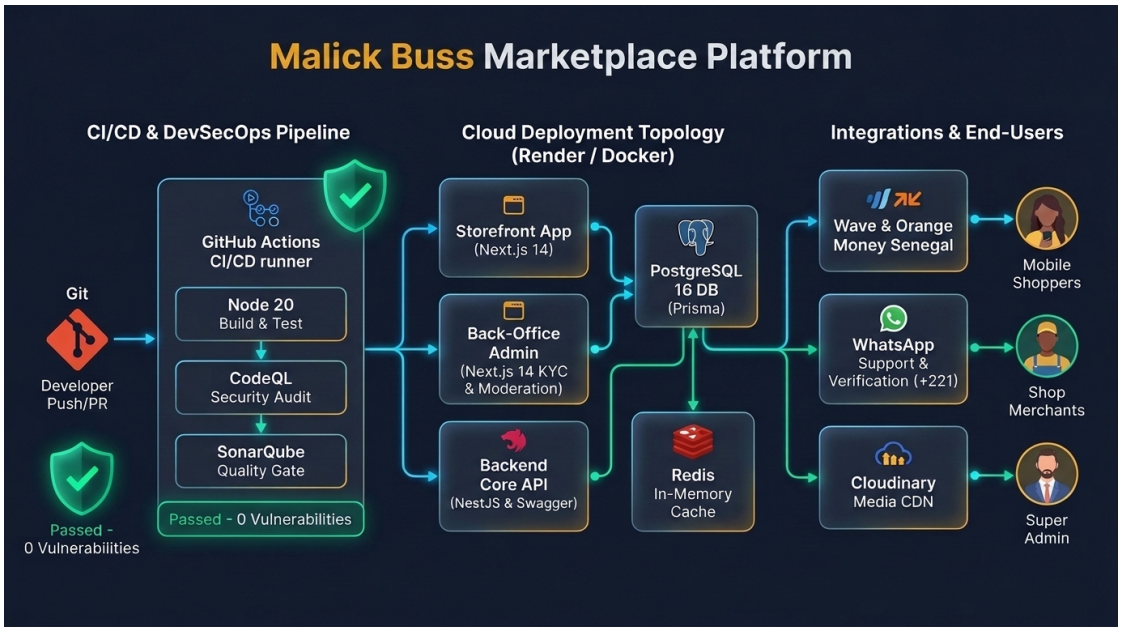


FIGURE 4.1 – Architecture Globale Cloud, CI/CD et Quality Gate SonarQube

Chapitre 5

DevOps, Déploiement Hybride & CI/CD

5.1 Stratégie Hybride : Render & Vercel

L'architecture de production dissocie la charge de travail pour une résilience maximale :

— **Render (Backend Core) :** Hébergement du service API NestJS et de la base managée PostgreSQL 16. La commande de pré-déploiement exécute automatiquement `npx prisma migrate deploy`.

— **Vercel (Frontends Edge) :** Déploiement de l'application cliente et de la console d'administration sur le réseau mondial Vercel Edge avec purge de cache instantanée.

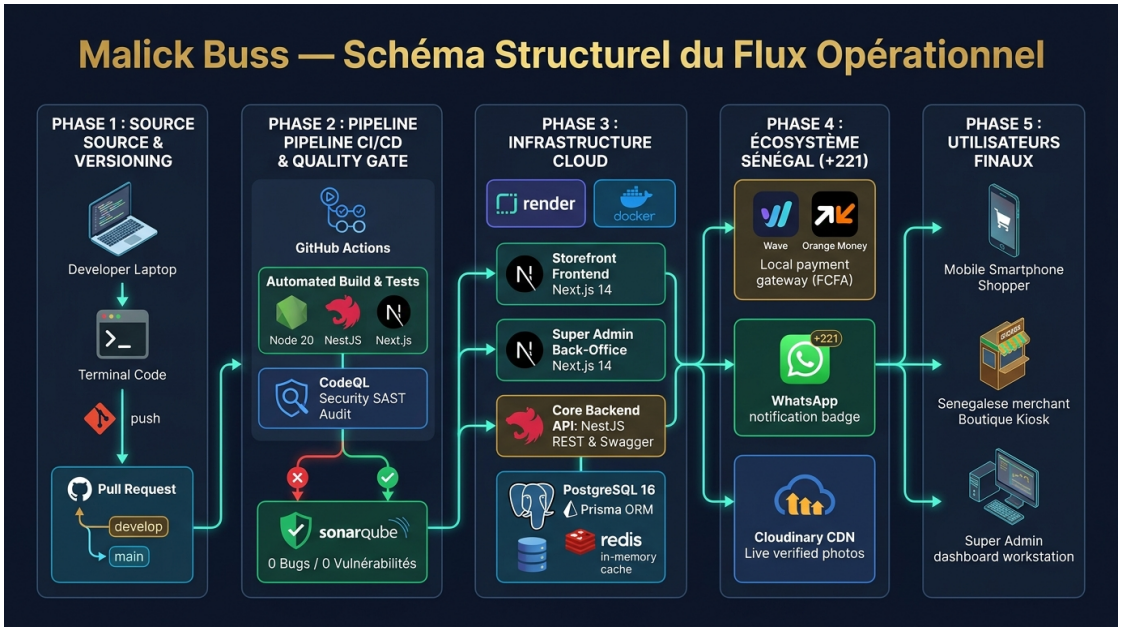


FIGURE 5.1 – Schéma Structurel du Flux Opérationnel en 5 Phases

5.2 Sondes de Santé HTTP

L'API expose deux points de contrôle cruciaux pour les orchestrateurs cloud :

— `GET /health/live` : Vérifie que le processus Node.js est actif et traite les requêtes.

— `GET /health/ready` : Exécute une requête `SELECT 1` sur PostgreSQL pour confirmer la disponibilité de la couche de données avant d'orienter le trafic client.

Chapitre 6

Conclusion & Manuel d'Exploitation

6.1 Commandes Usuelles pour l'Équipe

Commande npm	Effet / Action Opérationnelle
`npm run build:shared`	Compilation de la librairie partagée TypeScript.
`npm run build:backend`	Compilation NestJS de l'API de production.
`npm run build:frontend`	Génération statique des 42 pages de la vitrine client.
`npm run build:admin`	Génération statique de la console Super Admin.
`npm run test -workspace=backend`	Exécution des 8 tests unitaires de logique financière.
`npm run test:api -workspace=backend`	Exécution des 4 suites de tests d'intégration API.
`npm run test:coverage`	Génération du rapport de couverture LCOV pour SonarQube.
`npm run sonar:up`	Démarrage du serveur SonarQube Community en local via Docker.
`npm run doc:build`	Compilation du présent rapport technique en PDF haute fidélité.

Table 6.1 – Répertoire des commandes opérationnelles du projet

Document validé conforme par l'équipe d'architecture Malick Buss — Tous droits réservés © 2026.